StructuredPropertySelector コンポーネント UI 仕様書

1. 概要

1.1 コンポーネントの目的

StructuredPropertySelectorは、Placeノードのプロパティを構造別（基本情報/テンプレート/データ）に表示し、チェックボックスで選択できるダイアログコンポーネントです。Modal、Drawer、埋め込みの3つの表示モードをサポートします。

1.2 使用場面

- PetriNetViewerでPlaceノードをクリックした際のプロパティ選択（現在は使用されていない）
- SubNetViewerでのプロパティ選択（埋め込みモード）
- プロパティ選択ダイアログ（Modal/Drawerモード）

1.3 アクセス権限

- **必須ロール**: Experiment Admin（実験管理者）
- **認証**: JWT + Session Hybrid認証

---

2. 画面構成

2.1 レイアウト構造

2.1.1 Modalモード

```
┌──────────────────────────────────────────────────┐
│ プロパティ選択: [ノード名]                [×] 閉じる  │
├──────────────────────────────────────────────────┤
│ 選択中のプロパティ                                  │
│ ┌──────────────────────────────────────────┐   │
```

```
│ │ [基本情報: id] [テンプレート: temperature] ...   │   │
│ └──────────────────────────────────────────────┘   │
├──────────────────────────────────────────────────┤
│ [基本情報 2] [テンプレート 1] [データ]  ← タブ          │
├──────────────────────────────────────────────────┤
│ □  プロパティ名          値                        │
│ ──────────────────────────────────────────     │
│ ☑  ID                  place_001                 │
│ ☑  名前                サンプル材料                 │
│ □  タイプ              Material                  │
│                                                  │
└──────────────────────────────────────────────────┘
```

2.1.2 Drawerモード

```
                    ┌──────────────────┐
                    │ プロパティ選択       │
                    │ [ノード名]   [×]   │
                    ├──────────────────┤
                    │ 選択中のプロパティ    │
                    │ [基本情報: id] ...  │
                    ├──────────────────┤
                    │ [基本情報 2]        │
                    │ [テンプレート 1]    │
                    │ [データ]           │
                    ├──────────────────┤
                    │ □ ID  place_001   │
                    │ ☑ 名前 サンプル材料  │
                    │                  │
                    └──────────────────┘
```

2.1.3 埋め込みモード

```
┌──────────────────────────────────────────────────┐
│ プロパティ: [ノード名]                              │
├──────────────────────────────────────────────────┤
│ 選択中のプロパティ                                  │
│ [基本情報: id] [テンプレート: temperature] ...       │
├──────────────────────────────────────────────────┤
│ [基本情報 2] [テンプレート 1] [データ]  ← タブ          │
├──────────────────────────────────────────────────┤
│ □  ID                  place_001                 │
│ ☑  名前                サンプル材料                 │
│ □  タイプ              Material                  │
│                                                  │
└──────────────────────────────────────────────────┘
```

2.2 主要UI要素

2.2.1 ヘッダー

- **タイトル**: "プロパティ選択: [ノード名]"
- **閉じるボタン**: Modal/Drawerモードのみ

2.2.2 選択中のプロパティエリア

- **背景**: 白色、下部ボーダー
- **表示形式**: Tag（青色、クローズ可能）
- **フォーマット**: "[構造名]: [プロパティキー]"
- **空の場合**: Empty（「プロパティが選択されていません」）

2.2.3 タブ（選択数バッジ付き）

- **基本情報**: 基本情報プロパティ + 選択数バッジ
- **テンプレート**: テンプレートプロパティ + 選択数バッジ
- **データ**: データプロパティ + 選択数バッジ

2.2.4 プロパティテーブル

- **カラム**:
  - チェックボックス（50px幅、中央揃え）
  - プロパティ名（40%幅、太字）
  - 値（自動幅、グレー、12px）
- **サイズ**: small
- **スクロール**: 最大高さ400px
- **行クリック**: 行全体がクリック可能

---

3. 機能仕様

3.1 表示モード

3.1.1 モードの決定ロジック

```
const actualDisplayMode: 'modal' | 'drawer' | 'embedded' =
  displayMode || (embedded ? 'embedded' : 'modal');
```

- **displayMode指定あり**: displayModeを使用
- **embedded=true**: 'embedded'モード（後方互換性）
- **それ以外**: 'modal'モード（デフォルト）

3.1.2 Modalモード（解決策A）

```
<Modal
  title={`プロパティ選択: ${nodeName || nodeId || '未選択'}`}
  open={open}
  onCancel={onClose}
  onOk={onClose}
  width={800}
  style={{ top: 80 }}
  getContainer={getContainer || (() => document.body)}
  styles={{ body: { maxHeight: '70vh', overflow: 'hidden', display: 'flex', flexDirection:
'column' } }}
>
  {renderContent()}
</Modal>
```

- **幅**: 800px
- **位置**: top: 80px
- **コンテナ**: getContainerで指定、デフォルトはdocument.body

3.1.3 Drawerモード（解決策B）

```
<Drawer
  title={`プロパティ選択: ${nodeName || nodeId || '未選択'}`}
  placement="right"
  open={open}
```

```
  onClose={onClose}
  width={400}
  getContainer={getContainer || (() => document.body)}
  style={{ position: 'absolute' }}
  mask={true}
  maskClosable={true}
  destroyOnHidden={false}
>
  {renderContent()}
</Drawer>
```

- **幅**: 400px
- **配置**: 右側
- **コンテナ**: getContainerで指定
- **マスク**: 表示、クリックで閉じる

3.1.4 埋め込みモード

```
<div style={{
  display: 'flex',
  flexDirection: 'column',
  height: '100%',
  overflow: 'hidden',
  backgroundColor: '#fff'
}}>
  {/* ヘッダー */}
  <div style={{ padding: '16px', borderBottom: '1px solid #f0f0f0', flexShrink: 0 }}>
    <Text strong style={{ fontSize: 16 }}>
      {`プロパティ: ${nodeName || nodeId || '未選択'}`}
    </Text>
  </div>

  {/* コンテンツ */}
  <div style={{ flex: 1, overflow: 'auto' }}>
    {renderContent()}
  </div>
</div>
```

- **高さ**: 100%（親要素に依存）
- **オーバーフロー**: コンテンツエリアのみスクロール

3.2 プロパティ値のフォーマット

3.2.1 処理フロー

```
const formatPropertyValue = (propertyObj: PropertyObject | unknown): string => {
  let value;
  if (propertyObj && typeof propertyObj === 'object') {
    const obj = propertyObj as PropertyObject;
    if (obj.value !== undefined) {
      value = obj.value;
    } else if (obj.elements && obj.elements.value !== undefined) {
      value = obj.elements.value;
    } else if (obj.children && Array.isArray(obj.children) && obj.children.length > 0) {
      return `${obj.children.length}個の子要素`;
    }
  } else {
    value = propertyObj;
  }

  if (value === null || value === undefined || value === '') {
    return '未設定';
  }

  return String(value);
};
```

- **value優先**: obj.value → obj.elements.value → obj.children → propertyObj自体
- **子要素**: "N個の子要素"と表示
- **未設定**: "未設定"と表示

3.3 選択済みチップの表示

3.3.1 処理フロー

```
const renderSelectedChips = () => {
  const allSelected: Array<{
    structure: keyof StructuredProperties;
    property: string;
    label: string;
  }> = [];

  (Object.keys(structureLabels) as Array<keyof StructuredProperties>).forEach(structure => {
    if (selectedProperties[structure]) {
      selectedProperties[structure].forEach(prop => {
        allSelected.push({
```

```
          structure,
          property: prop,
          label: `${structureLabels[structure]}: ${prop}`
        });
      });
    }
  });

  if (allSelected.length === 0) {
    return (
      <Empty
        image={Empty.PRESENTED_IMAGE_SIMPLE}
        description="プロパティが選択されていません"
        style={{ margin: '16px 0' }}
      />
    );
  }

  return (
    <Space wrap style={{ padding: '16px' }}>
      {allSelected.map(({ structure, property, label }) => (
        <Tag
          key={`${structure}-${property}`}
          closable
          onClose={() => onPropertyToggle(structure, property)}
          color="blue"
        >
          {label}
        </Tag>
      ))}
    </Space>
  );
};
```

- **集約**: 全構造の選択プロパティを1つの配列に集約
- **表示**: 青色Tag、クローズ可能
- **フォーマット**: "[構造名]: [プロパティキー]"

---

4. データ仕様

4.1 Props定義

```
interface StructuredPropertySelectorProps {
  open: boolean;
  onClose: () => void;
  nodeId: string | null;
  nodeName?: string;
  structuredProperties: StructuredProperties;
  selectedProperties: SelectedProperties;
  onPropertyToggle: (structure: keyof StructuredProperties, property: string) => void;
  embedded?: boolean;  // 後方互換性のため残す
  displayMode?: 'modal' | 'drawer' | 'embedded';  // 新規
  getContainer?: () => HTMLElement;  // 新規
}
```

Props説明:

Prop名	型	必須	説明
open	boolean	✅	ダイアログの開閉状態
onClose	Function	✅	閉じる時のコールバック
nodeId	string | null	✅	ノードID
nodeName	string	-	ノード名
structuredProperties	StructuredProperties	✅	構造化されたプロパティ
selectedProperties	SelectedProperties	✅	選択状態
onPropertyToggle	Function	✅	プロパティ選択トグル時のコールバック
embedded	boolean	-	埋め込みモードフラグ（後方互換性）
displayMode	'modal' | 'drawer' | 'embedded'	-	表示モード（新規）
getContainer	Function	-	コンテナ要素取得関数（Modal/Drawer用）

4.2 型定義

4.2.1 PropertyObject

```
interface PropertyObject {
  key?: string;
  value?: unknown;
  type?: string;
  description?: string;
  elements?: {
    value?: unknown;
```

```
  };
  children?: unknown[];
}
```

4.2.2 StructuredProperties

```
interface StructuredProperties {
  basic: Record<string, PropertyObject>;
  template: Record<string, PropertyObject>;
  data: Record<string, PropertyObject>;
}
```

4.2.3 SelectedProperties

```
interface SelectedProperties {
  basic: string[];
  template: string[];
  data: string[];
}
```

4.3 State定義

```
const [activeTab, setActiveTab] = useState<string>('basic');
```

State説明:

State名	型	初期値	説明
activeTab	string	'basic'	アクティブなタブキー

---

5. API連携

5.1 使用するAPI一覧

#	API名	メソッド	エンドポイント	状態	説明
-	（なし）	-	-	-	API呼び出しなし

---

6. エラーハンドリング

6.1 エラーケース一覧

エラーケース	対処方法
structuredPropertiesが空	Empty（「[構造名]プロパティがありません」）表示
選択プロパティなし	Empty（「プロパティが選択されていません」）表示

---

7. 実装ファイル

7.1 ファイルパス

- **メインファイル**: `frontend/src/components/viewer/dialogs/StructuredPropertySelector.tsx`

7.2 依存ライブラリ

- `antd` : Modal, Drawer, Tabs, Table, Checkbox, Tag, Typography, Space, Empty
- `antd/es/table` : ColumnsType
- `@ant-design/icons` : CheckCircleOutlined, CloseCircleOutlined

---

8. 使用例

8.1 Modalモード

```
<StructuredPropertySelector
  open={modalOpen}
  onClose={() => setModalOpen(false)}
  nodeId="place_001"
  nodeName="サンプル材料"
  structuredProperties={structuredProperties}
  selectedProperties={selectedProperties}
  onPropertyToggle={handlePropertyToggle}
  displayMode="modal"
/>
```

8.2 Drawerモード

```
<StructuredPropertySelector
  open={drawerOpen}
  onClose={() => setDrawerOpen(false)}
  nodeId="place_001"
  nodeName="サンプル材料"
  structuredProperties={structuredProperties}
  selectedProperties={selectedProperties}
  onPropertyToggle={handlePropertyToggle}
  displayMode="drawer"
  getContainer={() => document.getElementById('petri-net-container')!}
/>
```

8.3 埋め込みモード

```
<div style={{ height: '600px' }}>
  <StructuredPropertySelector
    open={true}
    onClose={() => {}}
    nodeId="place_001"
    nodeName="サンプル材料"
    structuredProperties={structuredProperties}
    selectedProperties={selectedProperties}
    onPropertyToggle={handlePropertyToggle}
    displayMode="embedded"
  />
</div>
```

---

9. 変更履歴

日付	変更内容	担当者
2026-03-12	実装ベースで仕様書を再作成	Cline